

MODULE 5.1 · Concept 7 of 7

Building a ReAct Agent

Hands-on: implement a full ReAct agent with search, calculator, and code execution — 25 min lab

25 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000019

Prerequisite

AI-AG-IN-CD-000001

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — SETUP

From Theory to Code

Every concept you learned becomes 20 lines of Python

Lab Overview: What We're Building

A ReAct agent with 3 tools that can answer multi-step questions

Components:

LLM Brain	ChatOpenAI(model='gpt-4o-mini')	Reasoning engine — generates Thoughts, selects tools, produces Answers
Tool: web_search	Tool(func=search, desc='...')	Search the web for current information
Tool: calculator	Tool(func=calc, desc='...')	Evaluate mathematical expressions
Tool: code_executor	Tool(func=exec_code, desc='...')	Run Python code in sandbox
Agent Loop	create_react_agent(llm, tools)	Thought→Action→Observation→Repeat orchestration

Theory → Code Mapping: Perception=tool results, Reasoning=LLM, Action=tool calls, Loop=agent executor

Step 1: Environment Setup

Install dependencies, configure API key, import libraries

```
# Terminal
pip install langchain langchain-openai
export OPENAI_API_KEY="sk-..."

# Python imports
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain.tools import Tool
from langchain import hub
import os
```

That's it. LangChain handles the ReAct prompt template, tool execution, and loop management.

Step 2: Define Tool 1 – web_search

The search tool with schema and description

```
def web_search(query: str) -> str:
    """Search the web for current information."""
    # In production: call SerpAPI, Tavily, or Brave Search
    # For lab: use a simple wrapper
    from tavily import TavilyClient
    client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
    results = client.search(query, max_results=3)
    return str(results)

search_tool = Tool(
    name="web_search",
    func=web_search,
    description="Search the web for current information about events,\
                prices, people, or facts. Do NOT use for calculations."
)
```

Step 3: Define Tools 2 & 3 – Calculator + Code Executor

Two more tools for math and code execution

```
# Tool 2: Calculator
import ast, operator
def calculator(expression: str) -> str:
    """Evaluate math expression safely."""
    try:
        # Safe eval using ast
        result = eval(expression,
            {"__builtins__": {}},
            {"abs":abs, "round":round})
        return str(result)
    except Exception as e:
        return f"Error: {e}"

calc_tool = Tool(
    name="calculator",
    func=calculator,
    description="Evaluate a math\
    expression. Input: valid math\
    like '245 * 18.7'"
```

```
# Tool 3: Code Executor
import subprocess, tempfile
def code_executor(code: str) -> str:
    """Run Python code in sandbox."""
    with tempfile.NamedTemporaryFile(
        suffix=".py", mode="w",
        delete=False) as f:
        f.write(code)
        f.flush()
        result = subprocess.run(
            ["python", f.name],
            capture_output=True,
            text=True, timeout=30)
    return (result.stdout or "") +\
        (result.stderr or "")

code_tool = Tool(
    name="code_executor",
    func=code_executor,
    description="Run Python code.\
    Returns stdout + stderr.")
```

Step 4: Theory → Code Mapping

How agent anatomy maps to LangChain classes

Agent Concept	LangChain Class	What It Does
Perception	Tool result → context	Tool outputs injected as Observations
Memory	Context window (auto-managed)	Conversation history maintained by AgentExecutor
Reasoning	ChatOpenAI (LLM)	Generates Thoughts, decides Actions, produces Answers
Action	Tool(func=..., desc=...)	Wraps Python functions as callable agent tools
Goal	agent.invoke('user query')	The user query IS the goal
Loop	create_react_agent() + AgentExecutor	Orchestrates Thought→Action→Observation→Repeat

SECTION 2 — RUN

Agent in Action

Creating, running, and reading traces

Step 5: Create and Run the Agent

Three lines to create, wrap, and invoke

```
# Create the agent
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = hub.pull("hwchase17/react") # Standard ReAct prompt
tools = [search_tool, calc_tool, code_tool]

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(
    agent=agent, tools=tools,
    verbose=True,          # Print trace
    max_iterations=10,     # Prevent infinite loops
    handle_parsing_errors=True # Recover from bad JSON
)

# Run it!
result = executor.invoke({"input": "What is the population of Tokyo divided by 3?"})
print(result["output"])
```

Step 6: Reading the Agent Trace

verbose=True prints every Thought, Action, and Observation

1 **Thought** Thought: I need Tokyo's population. Let me search.

2 **Action** Action: web_search("Tokyo population 2024")

3 **Observation** Observation: 13.96 million (2024 census)

4 **Thought** Thought: Now I need to divide by 3.

5 **Action** Action: calculator("13960000 / 3")

6 **Observation** Observation: 4653333.33

7 **Final Answer** Answer: $\approx$ 4.65 million

Cost: 7 steps $\times$ $\sim$ 80 tokens avg = 560 tokens. At GPT-4o-mini: $\sim$ \$0.0004 per query.

Step 7: Worked Example 2 – Code Execution

Generate and run first 20 primes

Query: "Generate and run a Python script to find the first 20 primes"

Thought: I should write and execute Python code for this.

Action: `code_executor(code="def sieve(n):\n primes = []\n ...
 print(sieve(20))")`

Observation: `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]`

Thought: The code ran successfully and returned 20 primes.

Answer: The first 20 primes are: `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]`

Step 8: Implementing Stopping Conditions

Prevent infinite loops and runaway costs

```
executor = AgentExecutor(  
    agent=agent,  
    tools=tools,  
    verbose=True,  
    max_iterations=10,          # Hard stop after 10 loops  
    max_execution_time=60,      # Timeout after 60 seconds  
    early_stopping_method="force", # Return best attempt on timeout  
    handle_parsing_errors=True,  # Recover from bad JSON output  
    return_intermediate_steps=True # Include trace in output  
)
```

Without these guards, agents can loop forever or burn through your API budget. Always set max_iterations.

SECTION 3 — HARDEN

Error Handling and Logging

Making your agent production-ready

Step 9: Error Handling – When Tools Fail

Gracefully handle search failures, timeouts, and bad responses

```
def web_search_with_retry(query: str) -> str:
    """Search with retry on failure."""
    for attempt in range(3):
        try:
            result = tavily_client.search(query, max_results=3)
            if result["results"]:
                return str(result["results"][:3])
            # No results – reformulate
            query = f"{query} latest data" # Broaden search
        except Exception as e:
            if attempt == 2:
                return f"Search failed after 3 attempts: {e}"
            continue
    return "No results found. Try a different query."
```

Return errors as strings, not exceptions. The agent can reason about error messages and retry with different parameters.

Step 10: Trace Logging – Debug and Monitor

Print each step, count tokens, calculate costs

```
# Extract and log trace from return_intermediate_steps
result = executor.invoke({"input": query})

print(f"Answer: {result['output']}")
print(f"Steps: {len(result['intermediate_steps'])}")

total_tokens = 0
for i, (action, observation) in enumerate(result["intermediate_steps"]):
    print(f"  Step {i+1}:")
    print(f"    Tool: {action.tool}")
    print(f"    Input: {action.tool_input}")
    print(f"    Output: {observation[:100]}...")
    # Estimate tokens
    step_tokens = len(str(action.tool_input))/4 + len(str(observation))/4
    total_tokens += step_tokens

print(f"\nEstimated tokens: {int(total_tokens)}")
print(f"Estimated cost: ${total_tokens * 0.00000015:.6f}")
```

Step 11: Token Cost Calculation

What the worked examples actually cost

Query	Steps	Est. Tokens	Cost (GPT-4o-mini)	Time
"Tokyo population / 3"	7	~560	~\$0.0004	~2.5 sec
"First 20 primes"	5	~480	~\$0.0003	~3.0 sec
"Compare TCS vs Infosys revenue"	9	~1200	~\$0.0008	~5.0 sec
"Write market analysis of Indian IT"	15+	~3000	~\$0.002	~12 sec

Even complex queries cost less than \$0.01. A human researcher doing the same work: 5–30 minutes of labor.

Lab Exercises – Try These Yourself

Extend the basic agent with new tools and harder queries

Exercise 1

Add a `weather_lookup` tool

Ask: 'Should I carry an umbrella in Bangalore today?'

Easy

Exercise 2

Add a `code_executor` tool

Ask: 'Generate and run a script to find first 20 primes'

Medium

Exercise 3

Add error handling

What if search returns 'No results'? Implement retry with reformulated query.

Medium

Exercise 4

Add trace logging

Print each Thought/Action/Observation to a file. Count tokens per step. Plot usage.

Advanced

Key Takeaways

The essential points from this concept

- Three components to code:** tools (Tool class), brain (ChatOpenAI), loop (create_react_agent).
- Theory→code mapping is direct:** Perception=tool results, Reasoning=LLM, Action=tool calls.
- The agent trace is your debugging tool** — every Thought/Action/Observation is logged and inspectable.
- Cost per query: ~\$0.0004** for a multi-step reasoning task with GPT-4o-mini.
- This coding pattern is the foundation** for everything in Sub-Modules 5.3–5.7 and Module 9.

Next: Sub-Module 5.3 — Planning and Decomposition (Classical, LLM-based, and Hybrid Planning)

AI-AG-IN-CD-000001 Complete

Implementing ReAct Agents (Coding Lab)

Concept 7 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi